ATmega32A Module
GPIO (Digital Input / Output)

Ali Sahafi

1 Theory

GPIO (*General Purpose Input/Output*) is the most basic peripheral: each pin of the microcontroller can be a digital **output** (drive 0 V or 5 V) or a digital **input** (read whether the outside world applies 0 V or 5 V). The ATmega32A has four 8-bit ports: **PORTA**, **PORTB**, **PORTC** and **PORTD** — 32 pins in total.

Each port is controlled by three registers (x = A, B, C or D):

Register	Role	Bit meaning
`DDR`x	Data Direction	1 = output, 0 = input
`PORT`x	Output latch / pull-up	output: pin level; input: 1 = pull-up on
`PIN`x	Input read	the actual voltage currently on the pin

Internal pull-up. An unconnected input pin *floats*: it reads random values. For buttons, enable the internal pull-up (`INPUT_PULLUP`): the pin then rests at `HIGH`, and the button connects it to GND, so **pressed = LOW**. This "active-low" logic is the standard way to wire buttons.

2 API reference

The driver auto-corrects a common mistake: if you accidentally pass `PORTx` where `DDRx` is expected, `setDirection` fixes it for you.

Method	Description
`GPIO.setDirection(DDRx, pin, OUTPUT)`	Set single pin direction
`GPIO.setDirection(DDRx, ALL, INPUT_PULLUP)`	Set whole port direction
`GPIO.write(PORTx, pin, HIGH/LOW)`	Write single pin
`GPIO.write(PORTx, ALL, 0xF0)`	Write raw byte to whole port
`GPIO.write(PORTx, 0xF0)`	Shorthand whole-port write
`GPIO.read(PINx, pin)`	Read single pin — returns `HIGH` or `LOW`
`GPIO.read(PINx, ALL)`	Read whole port — returns 0–255
`GPIO.read(PINx)`	Shorthand whole-port read
`GPIO.toggle(PORTx, pin)`	Toggle single pin
`GPIO.toggle(PORTx, ALL)`	Toggle whole port

Constants: `INPUT`, `OUTPUT`, `INPUT_PULLUP`, `HIGH`, `LOW`, `ALL` (see Module 0).

3 Example: button-controlled LED

Wiring.

- Push button between **PD2** and **GND** (internal pull-up used, no external resistor needed).

- LED + 330 Ω resistor from **PC0** to GND (follows the button).
- LED + 330 Ω resistor from **PC1** to GND (blinks continuously).

Build & flash. `make gpio`

```
/*
 * GPIO Example -- button-controlled LED
 *
 * Wiring:
 *   - Push button between PD2 and GND (internal pull-up is used)
 *   - LED + 330R resistor from PC0 to GND
 *   - LED + 330R resistor from PC1 to GND (blinks continuously)
 *
 * Build & flash: make gpio
 */

#define F_CPU 8000000UL
#include "gpio.hpp"

int main() {
  GPIO.setDirection(DDRD, PD2, INPUT_PULLUP); // button (reads LOW when pressed)
  GPIO.setDirection(DDRC, PC0, OUTPUT);   // LED follows the button
  GPIO.setDirection(DDRC, PC1, OUTPUT);   // LED blinks on its own

  while (true) {
    // Button is active-low: pressed = LOW -> turn LED on
    if (GPIO.read(PIND, PD2) == LOW) {
      GPIO.write(PORTC, PC0, HIGH);
    } else {
      GPIO.write(PORTC, PC0, LOW);
    }

    GPIO.toggle(PORTC, PC1);
    _delay_ms(200);
  }

  return 0;
}
```

Note that reading uses `PIND` (the input register) while writing uses `PORTC` (the output latch) — mixing these up is the classic GPIO bug.

4 Exercises

1. Change the program so the LED on PC0 *toggles* once per button press instead of following it. (Hint: you must detect the *edge* — remember the previous button state in a variable.)
2. Connect 8 LEDs to PORTC and write a "running light" (Knight Rider) pattern using the whole-port `GPIO.write(PORTC, value)` overload.
3. Why does a button need a pull-up at all? Remove `INPUT_PULLUP` (use plain `INPUT`), run the program and explain what you observe.